TECHNICAL REQUIREMENTS DOCUMENT (TRD)

CaféNest — Cafe Management and Online Ordering System

Document Version: 1.0
Project Type: College DBMS Lab Project
Team: Om Watane, Ambika Shelke, Tejaswani, and one additional team member

Mandatory Technology Stack

- **Frontend:** HTML, CSS, and JavaScript only.
- **Backend:** Python only.
- **Database:** SQL for permanent data storage.
- **Version Control:** Git and GitHub.

Strict restriction: Do not use PHP, Node.js, Express.js, Java, C#, TypeScript, React, Angular, Vue, or any other prohibited programming language or frontend framework. Do not create a second backend. Do not replace SQL with JSON files, browser storage, or temporary in-memory data.

Prepared as a technical planning document. Requirements and technical details should be checked against the final implementation before submission.

---

1. Technical Overview

CaféNest is planned as a browser-based application with a customer interface, an administrator/staff interface, a Python backend, and a relational SQL database.

The backend validates requests, enforces business rules, and stores application records. The frontend communicates with the Python backend through API requests. The backend uses SQL queries to read and update persistent data.

The implementation must use only HTML, CSS, JavaScript, Python, and SQL for the application. The selected Python web framework and SQL database engine should be confirmed before implementation if not already specified.

2. Technology Stack

Layer	Technology	Purpose
Frontend structure	HTML	Website pages and content
Styling	CSS	Layout, visual design, and responsive styling
Client-side behavior	Vanilla JavaScript and Fetch API	Form handling and asynchronous API requests
Backend	Python	Validation, sessions, business logic, and API responses

Database	SQL	Persistent relational data and integrity constraints
Version control	Git and GitHub	Source control and collaboration
Local environment	Python runtime and approved SQL database environment	Local testing and demonstration

No other programming language or frontend framework is permitted.

3. System Architecture

3.1 Request Flow

1. The user interacts with a customer or admin page in the browser.
2. JavaScript sends requests to Python API endpoints using Fetch API.
3. Python checks the session, validates input, applies business rules, and queries the SQL database.
4. The endpoint returns a consistent response.
5. The frontend updates the page and shows success, loading, or error feedback.

3.2 Main Components

- **Customer UI:** Menu, item details, cart, checkout, profile, and order history.
- **Admin/staff UI:** Dashboard, menu management, orders, tables, inventory, suppliers, and staff.
- **API layer:** Authentication, menu, cart/checkout, orders, payments, and administration endpoints.
- **Database layer:** Relational tables, keys, indexes, constraints, and transactions.

4. Database Design

The proposed schema separates customers, menu data, orders, payments, tables, staff, and inventory records. Use primary keys, foreign keys, uniqueness rules, and indexes to protect data integrity.

Table	Purpose	Example key fields
customers	Customer accounts and profile	customer_id, name, email, password_hash
categories	Menu categories	category_id, category_name
menu	Food and beverage items	menu_id, category_id, name, price, is_available
orders	Order header and customer/order status	order_id, customer_id, table_id, status, total
order_details	Items belonging to an order	order_detail_id, order_id, menu_id, quantity, unit_price
payments	Payment method and simulated payment state	payment_id, order_id, amount, method, status
cafe_tables	Dine-in table and availability	table_id, table_number, status

staff	Staff accounts and role information	staff_id, name, role, login fields
suppliers	Supplier contact records	supplier_id, supplier_name, contact
inventory	Stock item and quantity	inventory_id, supplier_id, item_name, quantity, reorder_level
stock_movements	Stock additions and deductions	movement_id, inventory_id, quantity_change, reason
menu_price_audit	History of menu price changes	audit_id, menu_id, old_price, new_price, changed_at

The proposed design contains 12 core tables. Table fields may be refined during implementation while preserving the required relationships and project scope.

5. Database Relationships

- One category can contain many menu items.
- One customer can place many orders.
- One order can contain many order_details records.
- Each order detail refers to one menu item.
- An order may have one or more payment records depending on the chosen design; enforce the intended rule with constraints.
- A cafe table can be linked to dine-in orders when the dine-in flow is enabled.
- A supplier can be associated with inventory items.
- One inventory item can have many stock movement records.
- Menu price audit records refer to the menu item whose price changed.

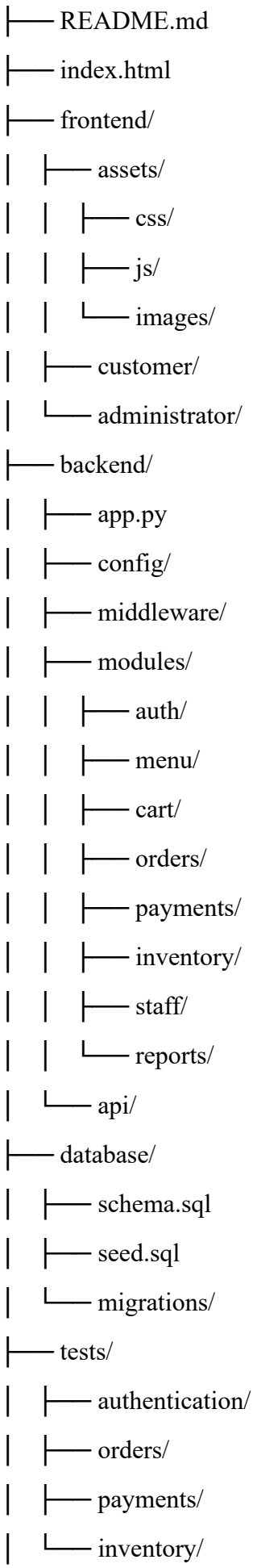
6. Backend Modules and Suggested Folder Structure

6.1 Modules

- Authentication and session handling.
- Menu/category read and administration CRUD operations.
- Cart validation and order placement.
- Order listing and status updates.
- Simulated payment record creation.
- Table management.
- Inventory, stock movement, and supplier management.
- Staff/role management and dashboard reports.

6.2 Suggested Project Structure

CafeNest/


├── README.md
├── index.html
├── frontend/
│ ├── assets/
│ │ ├── css/
│ │ ├── js/
│ │ └── images/
│ ├── customer/
│ └── administrator/
├── backend/
│ ├── app.py
│ ├── config/
│ ├── middleware/
│ ├── modules/
│ │ ├── auth/
│ │ ├── menu/
│ │ ├── cart/
│ │ ├── orders/
│ │ ├── payments/
│ │ ├── inventory/
│ │ ├── staff/
│ │ └── reports/
│ └── api/
├── database/
│ ├── schema.sql
│ ├── seed.sql
│ └── migrations/
├── tests/
│ ├── authentication/
│ ├── orders/
│ ├── payments/
│ └── inventory/

```
└── docs/
    ├── PRD.md
    ├── TRD.md
    └── database-design.md
```

The exact folder structure can be adjusted during development, but frontend, backend, database scripts, tests, and documentation should remain clearly separated. Python source files must contain the server-side logic. SQL files must define database structures and seed data.

7. API Design Guidelines

Endpoints should return consistent responses with predictable fields such as success, message, and data. Use suitable HTTP status codes and do not return stack traces or credentials.

Endpoint example	Method	Purpose
/api/auth/register	POST	Create a customer account
/api/auth/login	POST	Authenticate and start a session
/api/auth/logout	POST	End the current session
/api/menu	GET	List available menu items
/api/menu/save	POST	Create or update a menu item (admin only)
/api/orders	POST	Validate cart and place an order
/api/orders	GET	List orders allowed for the current user
/api/orders/update-status	POST	Update order status (authorized staff/admin)
/api/payments/demo	POST	Save a simulated payment result
/api/admin/dashboard	GET	Return dashboard summary figures

These are proposed endpoint paths. They must be implemented, validated, and tested before being considered functional.

Example API response:

```
{
  "success": true,
  "message": "Order created successfully",
  "data": {
    "order_id": 101,
    "order_status": "Pending"
  }
}
```

API responses may use JSON as a data exchange format. JSON files must not replace the SQL database as permanent storage.

8. Business Rules

- The Python backend must calculate order totals using trusted menu prices; do not trust totals submitted by the browser.
- Reject empty carts, invalid item IDs, non-positive quantities, and unavailable items.
- Save the order header and its line items consistently in a database transaction.
- Recheck item price and availability during checkout.
- Customers may access only their own profiles and orders; staff/admin access must be role-checked.
- Only valid order status transitions should be accepted.
- Simulated payment status must be clearly labelled as simulated and must not imply that money was transferred.
- Stock movements should record a reason and quantity change; update inventory consistently.
- Deactivation is often safer than deleting menu items that are referenced by past orders.

9. Security and Data Integrity

- Use secure password hashing supported by the Python implementation.
- Use parameterized SQL queries.
- Validate and normalize all incoming values on the server.
- Use session checks and role-based authorization for protected endpoints.
- Regenerate session identifiers after login and use secure cookie settings where supported.
- Escape user-generated text when rendering HTML to reduce cross-site scripting risk.
- Protect state-changing requests against CSRF where applicable.
- Keep database credentials out of public repositories and use environment-specific configuration.
- Use transactions for order creation and multi-step stock updates.
- Do not log passwords, session tokens, or sensitive payment data.

10. Error Handling and API Responses

- Return clear validation messages for missing or invalid fields.
- Use consistent response formats across endpoints.
- Handle unauthenticated and unauthorized requests with suitable status codes.
- Log technical details privately while showing users a short, safe error message.
- Ensure frontend JavaScript handles failed requests and malformed responses without breaking the page.

11. Testing Plan

Test area	Example checks
Authentication	Valid/invalid login, duplicate email, logout, session protection
Menu	List items, show images, update price, disable unavailable item
Cart	Add/remove item, update quantity, handle empty cart
Checkout	Correct total, invalid quantity rejection, unavailable item rejection
Orders	Order and detail rows saved, order history permissions, status updates
Payments	Cash/simulated UPI record, status labelled accurately
Admin access	Customer cannot access admin-only endpoints
Inventory	Stock update, movement history, low-stock warning
UI and integration	Links/buttons work, loading/error states, responsive layout

12. Deployment and Setup

Local Development

- Install and configure Python.
- Set up the approved SQL database environment.
- Create the database and execute the SQL schema and optional seed scripts.
- Configure database connection values outside public source files.
- Start the Python backend using the documented run command.
- Open the application entry page in a browser.
- Test customer and admin flows using separate sessions.
- Document prerequisites, startup commands, test credentials (if any), and known limitations in README.md.
- Before deployment, verify that uploaded images, API paths, and session behavior work in the target environment.

Production Deployment

- Deploy the Python backend to a compatible hosting environment.
- Configure SQL database credentials securely and use HTTPS.
- Disable detailed error output for visitors.
- Configure database backups.
- Verify file permissions and session settings.
- Test authentication, ordering, payment simulation, and administration after deployment.

The application must not rely on hardcoded local paths that only work on one computer.

13. Development Phases

- **Phase 1 — Planning and design:** Finalize requirements, prepare wireframes, create database ER diagrams, define user roles and business rules, and approve the PRD and TRD.
- **Phase 2 — Database and project setup:** Create the project directory structure, configure Python and the SQL database, write schema and seed scripts, and add keys, indexes, and constraints.
- **Phase 3 — Frontend development:** Create customer and administrator interfaces, implement responsive layouts, and add food images and consistent design.
- **Phase 4 — Backend development:** Implement authentication, menu and category operations, cart, checkout, orders, simulated payments, inventory, supplier and staff modules, audit records, and reports using Python.
- **Phase 5 — Integration and testing:** Connect frontend forms to Python endpoints, test SQL database operations, fix validation and authorization issues, and verify calculations and reports.
- **Phase 6 — Deployment and demonstration:** Prepare setup instructions, configure the final environment, load demonstration data, test the application, and prepare the final report, ER diagram, and presentation.

14. Team Responsibilities

Member	Suggested responsibility
Om Watane	Backend logic, Python API integration, validation, order/checkout flow, and project integration
Ambika Shelke	Frontend layout, customer/admin pages, styling, and responsive behavior
Tejaswani	Testing, integration checks, data validation, and documentation support
Fourth team member	Database design, seed data, admin/inventory module, or another agreed module

Update the responsibility table to match the actual work completed by each team member before submission.

Shared responsibilities:

- Requirements review and design decisions.
- Git and GitHub collaboration.
- Code reviews, integration, and defect fixing.
- Documentation, testing, and final demonstration.
- Understanding the overall architecture and database design.

Contributions should be tracked through actual code commits, reviews, testing records, and documentation.

15. Deliverables and Completion Checklist

- Frontend pages load and navigation works.

- Python endpoints validate input and return predictable responses.
- Database schema and relationships match the implemented features.
- Customer can place an order and view its status.
- Admin can manage the agreed menu/order functions.
- Simulated payment is clearly marked and never presented as a real transaction.
- Core negative tests and role checks pass.
- README contains setup instructions and known limitations.
- The PRD/TRD are revised so they describe the final implementation accurately.

16. Future Enhancements

- Real payment gateway integration after appropriate security and compliance review.
- Email/SMS order notifications.
- Ingredient-level stock deduction based on menu recipes.
- Customer ratings, feedback, and loyalty points.
- Advanced sales analytics and exportable reports.

17. Mandatory Technology Restrictions

1. Frontend code must use only HTML, CSS, and vanilla JavaScript.
2. Backend logic and APIs must use Python only.
3. Persistent application data must be stored in an SQL database.
4. Do not use PHP, Node.js, Express.js, Java, C#, TypeScript, React, Angular, Vue, or any other prohibited programming language or frontend framework.
5. Do not create a second backend in another language.
6. Do not use JSON files, browser localStorage, sessionStorage, or temporary in-memory data as the permanent database.
7. If an older document mentions PHP, replace that implementation requirement with Python.
8. Do not mark features complete until they have been implemented and tested.

Final Project Definition

CaféNest is a full-stack cafe management and online ordering system built with HTML, CSS, JavaScript, Python, and SQL. It combines customer ordering, menu management, order processing, simulated payments, cafe table management, employee records, supplier management, inventory tracking, stock movement history, menu price auditing, and administrative reporting.

The project is planned around a modular architecture, a relational database with 12 core tables, server-side validation, secure authentication, transaction-based order processing, and documented testing and deployment procedures.

Implementation note: This TRD defines the intended system. Features and database tables should be marked complete only after they have been implemented and verified in the actual application.